GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY

Faculty of Computing - Intake 42



GROUP PROJECT REPORT

AI-Based Human Body Fat Percentage Measuring System

Group Members

Name	Index Number	Degree Program
S.M. Kuruppu	Computer Science	D/BCS/25/0006
M.H.K.S. Rajapaksha	Information Technology	D/BIT/24/0028
K.L.C. Jeewananda	Information Systems	D/BIS/24/0039
SY Ganegoda	Data Science & Business Analysis	D/DBA/25/0033
Prinjal Oli	Software Engineering	7176

Abstract

Body fat percentage is a clinically superior measure of health risk than Body Mass Index (BMI), but calculating Body fat percentage usually requires expensive technologies such as DXA or underwater weighing to calculate precisely. With regards to this problem, this project introduces AuraFat.AI, a machine learning model which predicts body fat percentage from 9 easy self measurable features, such as age, gender, weight, height, BMI, and circumference measurements like waist, arm, calf, and thighs. The AI model was trained using the 2005-2006 wave of the National Health and Nutrition Examination Survey, which provided a nationally representative sample of the US population, including DXA-measured body composition for comparison with the predicted values. After data cleaning and combining three separate NHANES source files, the resulting pool contained 5,628 individuals and 11 columns of data.

We had tested and evaluated three types of models on a test set of 845 individuals: a Linear Regression, a full and a reduced random forest (using 9 and 6 features), a neural network implemented in PyTorch. After comparing the results the neural network achieved the best results with a Mean Absolute Error (MAE) of 2.55 percentage points, Root Mean Squared Error (RMSE) of 3.23, and an R-Squared (R^2) of 0.877 on the test set. It was closely followed by the Random Forest (MAE 2.63, R^2 0.865), and Linear Regression (MAE 3.23, R^2 0.804) showed the best performance among the three types of models. However, the neural network required significantly more engineering and tuning to achieve this performance, including feature scaling, hundreds of training epochs, early stopping checkpointing, and fixing a bug with referencing state_dict(). In terms of practical applications and contributions to the field, the Random Forest's comparable performance (with the expected range of error for circumference body fat prediction of ±3–4 percentage points against DXA) and significant simplicity has a greater value.

Table of Contents

1. Introduction 1
   1.1 Context and Motivation 1
   1.2 The Limitations of BMI as a Proxy 1
   1.3 Clinical Gold Standards and the Accessibility Gap 1
   1.4 Machine Learning as a Bridge 1
   1.5 Project Scope and System Name 1
2. Problem Background 2
   2.1 The Global Obesity Burden 2
   2.2 Why BMI Is Insufficient 2
   2.3 DXA as Ground Truth 2
   2.4 The Accessibility Gap in Practice 2
   2.5 Prior Work and Research Precedent 2
   2.6 The Specific Problem This Project Addresses 2
3. Problem Statement 3
4. Objectives 4
5. Selected AI Techniques 5
   5.1 Linear Regression (Baseline) 5
   5.2 Random Forest Regressor 5
   5.3 Feedforward Neural Network (BodyFatNet) 5
   5.4 Data Preprocessing 6
   5.5 Model Selection and Validation Strategy 6
6. Justification of AI Techniques 7
   6.1 A Neural Network for a Non-Linear Regression Problem 7
   6.2 PyTorch and Google Colab as the Development Environment 7
   6.3 NHANES DXA Data as the Training Source 7
   6.4 Classical Baselines as a Rigour Check 7
   6.5 Feature Selection Rationale 7
   6.6 Handling of the Validation Set 8
7. System Design 9
   7.1 End-to-End Workflow 9
   7.2 Data Pipeline Design 9
   7.3 Neural Network Architecture Design 9
   7.4 Deployment Architecture 10
   7.5 Repository Structure (GitHub) 10
8. Implementation Details 11
   8.1 Development Environment and Tooling 11
   8.2 Data Pipeline Implementation 11
   8.3 Data Splitting and Preprocessing 12

8.4 Neural Network Implementation (BodyFatNet) 12
8.5 Training Loop and Checkpointing 13
8.6 Model Evaluation and Persistence 14
8.7 Inference Server 14
8.8 Engineering Challenges and Resolutions 15
9. Testing and Results 16
9.1 Evaluation Metrics 16
9.2 Final Model Comparison 16
9.3 Neural Network Training Dynamics 17
9.4 Feature Importance Analysis 18
9.5 Discussion of Results 19
10. Limitations and Discussion 20
10.1 The Accuracy versus Reproducibility Trade-off 20
10.2 Feature Set Limitations 20
10.3 Deferred Landmark Measurements 20
10.4 Validation Methodology 20
10.5 Dataset Scope 21
10.6 Error Margins in Practice 21
11. Ethical and Social Considerations 22
11.1 Health Equity and Accessibility 22
11.2 Sensitive Attributes and Algorithmic Bias 22
11.3 Data Privacy and Consent 22
11.4 Psychological and Social Impact 22
11.5 Transparency and Reproducibility 22
12. Conclusion 23
12.1 Future Work 23
References 24

1. Introduction

1.1 Context and Motivation

Obesity and excess adiposity are among the most biggest public health challenges of the modern era. The World Health Organization (WHO) estimates that 2.5 billion adults were diagnosed overweight in 2022, 890 million of whom were living with obesity, and that adult obesity prevalence has doubled since 1990 . Excess body fat is causally linked to cardiovascular disease, type 2 diabetes, several forms of cancer and a range of metabolic disorders.

1.2 The Limitations of BMI as a Proxy

Body mass index (BMI), based on height and weight only, is currently the most widely used screening proxy for adiposity because of its low cost and speed of measurement with no requirement for special equipment. However, BMI has one significant structural limitation: it cannot differentiate between fat mass and lean mass Highly muscular individuals can be misclassified as overweight or obese, while people with low muscle mass and a “normal” weight can carry excess fat without it being apparent. Body fat percentage the proportion of total body mass made up of fat tissue is a more direct and clinically meaningful measure, but it is significantly harder to obtain.

1.3 Clinical Gold Standards and the Accessibility Gap

The clinical standards for body fat percentage measurement are hydrostatic (underwater) weighing and dual-energy X-ray absorptiometry (DXA). The latter, in particular, has become a gold standard in body composition analysis, being both accurate and reliable . At the same time, such procedures are usually only available in research and clinical settings, meanwhile, Less accurate, but more accessible methods like bioelectrical impedance scales which, while easy to use and relatively inexpensive, may significantly vary in readings depending on the user.

This creates a persistent accessibility gap: **clinically accurate methods are inconvenient, while convenient methods are clinically inaccurate.**

1.4 Machine Learning as a Bridge

Machine learning offers a way across this gap. It has been noted through prior work that artificial neural networks can predict body fat percentage from simple, easily obtained anthropometric and laboratory measurements more accurately than classic regression models. This project builds directly on that line of work by training a neural network on the National Health and Nutrition Examination Survey (NHANES)

1.5 Project Scope and System Name

The resulting system, **AuraFat.AI**, predicts body fat percentage from nine anthropometric inputs and places the result in a body composition category. It is built on Python, using PyTorch library for the neural network, scikit-learn for the classical baselines and Google Colab for training. A lightweight web interface has been prepared.

2. Problem Background

2.1 The Global Obesity Burden

The scale of the global obesity problem provides the epidemiological context for this project. The health consequences of excess adiposity are severe because obesity substantially raises the risk of cardiovascular disease, type 2 diabetes, hypertension, dyslipidaemia and certain cancers and musculoskeletal disorders.

2.2 Why BMI Is Insufficient

BMI is calculated by dividing the kilograms by the meter squared. The main advantage of calculating BMI is that this calculation is relatively simple and only requires a person's height and weight. These two parameters can be ascertained with a scale and a stadiometer, which are mostly available. The primary disadvantage of BMI is that it reflects only the ratio between weight and height. BMI does not take into account the ratio between body fat and muscle mass. Thus, BMI does not differentiate between lean mass and adipose tissue, and two people with the same BMI can have completely different body composition. normal BMI can significantly underestimate body fat percentage in some people. In other words, BMI will classifies people with muscle mass as overweight.

2.3 DXA as Ground Truth

DXA uses two X-ray beams with differing energy levels to penetrate the body, and measures the degree to which bone, lean tissue and fat tissue attenuate each beam. This yields a region-specific, accurate estimate of fat mass, lean mass and bone mineral content. In cross-method validation studies, DXA is found to be significantly more accurate and reproducible than bioelectrical impedance analysis and is therefore used as the ground-truth source in this project. NHANES dataset provides DXA-measured total body fat percentage (DXDTOPF)

2.4 The Accessibility Gap in Practice

In practice, a person who wants to know their body fat percentage faces a constrained choice. They can visit a clinic or research facility for DXA or underwater weighing which can be accurate but costly, Or they can use a consumer BIA scale or skinfold calipers cheap and accessible, but sensitive to hydration, measurement technique and device calibration drift. There is no low cost tool that estimates body fat percentage at clinical-grade accuracy from, self-measurable inputs.

2.5 Prior Work and Research Precedent

The most directly relevant prior work is that of Ferenci and Kovács, who showed that artificial neural networks trained on anthropometric and laboratory measurements outperform classical regression-based body fat formulas on the same class of features. Their study provides both the methodological precedent and the performance benchmark for this project. Additional context comes from Kelly et al, who established DXA body composition reference values from NHANES, and from Johnson, whose classic 252-subject body fat dataset labelled using the Siri body-density equation serves as an independent external benchmark for later validation.

2.6 The Specific Problem This Project Addresses

There is currently no widely available, low cost option for estimating one's body fat percentage based on easily measurable factors accurately enough for medical purposes, which doesn't require a professional procedure such as a DXA scan or hydrostatic weighing..

3. Problem Statement

The formal problem statement for this project is:

"How can an AI based system trained on clinically verified body composition data estimate human body fat percentage accurately from simple, accessible input measurements, so that individuals and health practitioners can obtain a reliable body composition estimate without access to specialised imaging equipment?"

ThIS problem has five defining characteristics:

1. **It is a supervised regression problem**.
2. **It is a tabular problem.**
3. **It has a hard accessibility constraint.**
4. **It has an accuracy target.**
5. **It must be deployable.**

The problem statement remained substantively unchanged between the project proposal and the final implementation. What evolved was the choice of a single NHANES cycle, the finalisation of the nine feature input set, and the formal inclusion of the neural network.

4. Objectives

The project was guided by six concrete objectives, defined at the proposal stage and tracked through implementation.

Objective 1: Data Acquisition and Preparation. To obtain, clean and combine the NHANES Demographics (DEMO_D), Body Measures (BMX_D) and DXA whole body composition (DXX_D) files into one labelled data set that would be appropriate for supervised learning. The final data set had to be internally consistent, without any biologically implausible records and properly documented for reproducibility.

Objective 2: Neural Network Development. Design and train a feedforward artificial neural network in PyTorch on Google Colab that could estimate body fat percentage based on age, sex, and body measurements. The network had to be not too large so that it did not overfit the data, and its training had to be reproducible.

Objective 3: Classical Baseline Implementation. Implement and train classical baselines, Multiple Linear Regression and a Random Forest, against which the neural network would be compared.

Objective 4: Rigorous Evaluation. To evaluate all models against DXA-measured ground truth using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and the coefficient of determination (R^2), and to select the best-performing model.

Objective 5: External Validation. To test the selected model on an independent benchmark, the classic 252-subject Johnson body fat dataset with Siri-equation labels

Objective 6: Prototype Deployment. To build a simple prototype interface that accepts user-entered measurements and returns a predicted body fat percentage together with a body composition category.

Table 4.1 summarises the outcomes so far.

No.	Status	Outcome
1	Achieved	Three NHANES files cleaned and merged into a 5,628-row, 11-column dataset.
2	Achieved	BodyFatNet trained in PyTorch with a fixed seed and validation-based checkpointing.
3	Achieved	Linear Regression and Random Forest (9- and 6-feature) baselines trained.
4	Achieved	All models evaluated on MAE, RMSE and R^2; the neural network performed best.
5	Deferred	moved to future work.
6	Partially achieved	Frontend and Python inference server has been built and final polish and user testing remain.

Table 4.1 — Status of project objectives.

5. Selected AI Techniques

The project implements and compares three model families, ranging from classical statistical learning to deep learning. Each model is trained on the same cleaned dataset, so that the results are directly comparable in terms of data and features.

5.1 Linear Regression (Baseline)

Multiple Linear Regression is the most basic method available to us. It assumes that a linear function can capture the relationship between our nine inputs and body fat percentage. It most likely will not capture the true relationship, as it is most probably non-linear, but will set the bar for the lowest acceptable accuracy. If a more complex neural network does not have better accuracy than a simple linear regression, it is not worth pursuing. Luckily, scikit-learn provides us with an implementation of multiple linear regression, which we can access through its LinearRegression class.

5.2 Random Forest Regressor

The Random Forest Regressor is a method that builds many decision trees during training and outputs the mean of their predictions. It is a strong baseline for tabular data because it handles non-linear relationships natively, needs no feature scaling and reports feature importance out of the box. Two variants were trained:

1. **Full 9-feature model.**
2. **Reduced 6-feature model**.

5.3 Feedforward Neural Network (BodyFatNet)

The primary model is a fully connected feedforward network (multilayer perceptron) implemented in PyTorch. The architecture, named **BodyFatNet**, is deliberately small to avoid overfitting the training set of 3,938 rows. Its layers are summarised in Table 5.1

Layer	Neurons	Activation	Role
Input	9	—	One neuron per standardised input feature
Hidden layer 1	64	ReLU	Learns non-linear feature interactions
Hidden layer 2	32	ReLU	Builds higher-level representations
Output	1	Linear	Predicted body fat percentage

Table 5.1 — BodyFatNet architecture (2,753 trainable parameters).

The training configuration is given in Table 5.2.

Setting	Value
Loss function	Mean Squared Error (MSE)
Optimiser	Adam, learning rate 0.001
Batch size	Full batch (one update per epoch on the whole training set)
Epochs	Up to 5,000, with the best checkpoint kept (early-stopping logic)
Random seed	`torch.manual_seed(42)` for deterministic, reproducible training
Preprocessing	`StandardScaler` fitted on the training data only

Table 5.2 — Neural network training configuration.

Features are standardised with `StandardScaler`, which is fitted **only** on the training data and then applied to the validation and test data to prevent data leakage. Scaling is essential for the neural network because gradient descent is sensitive to feature scale, whereas tree-based models are scale-invariant.

5.4 Data Preprocessing

All pipelines share the same data preparation steps:

- **Missing values:** rows with a missing value in any of the final selected columns are dropped (only 12–57 rows per column were affected).
- **Multiple imputation:** the DXA file contains five multiple-imputation rows per person. After filtering to completed exams, these are averaged into a single row per person using `groupby('SEQN').mean()`.
- **Standardisation:** applied only for the neural network, fitted on the training split only.
- **Derived features:** BMI is taken directly from NHANES (`BMXBMI`), although it is technically derived from weight and height.

5.5 Model Selection and Validation Strategy

The evaluation strategy was designed for honest performance reporting:

- **Baseline models** (Linear Regression and Random Forests) were trained on an 80/20 train–test split with `random_state=42` and have no hyperparameter search beyond their defaults.
- **The final neural network** uses a 70/15/15 train–validation–test split.
- **Architecture selection** was supported by a systematic layer-increment experiment (notebook 05, Section 7.5) rather than an arbitrary choice.
- **Early stopping:** training runs for up to 5,000 epochs, and the weights are checkpointed whenever the validation loss reaches a new minimum.
- **Metrics:** MAE, RMSE and R^2, all computed against DXA ground truth.

6. Justification of AI Techniques

The choice of techniques is grounded in both the physiological nature of the problem and the practical constraints of the project.

6.1 A Neural Network for a Non-Linear Regression Problem

The body fat percentage of an individual is not linear with respect to simple anthropometric inputs across the range of the body types. Such measures as waist circumference and body fat are related differently in the case of males and females, while the body mass index (BMI) and waist circumference have a non-linear interaction. Thus, the simplest linear model would fail in capturing such relations. Ferenci and Kovács [3] have shown that neural networks outperform the classical regression formulas for the case of such features, thus directly motivating the choice of neural networks as the main instrument in this study. The two hidden layers provide sufficient depth to learn the relationships without incurring the risk of overfitting due to being too large.

6.2 PyTorch and Google Colab as the Development Environment

The software PyTorch was selected because it is a popular and well-documented framework which utilizes a dynamic approach for the construction of the computational graphs which is more suitable for the iterative development process. The possibility to alter the architecture during the training was particularly beneficial for the current task, as it allowed to experiment with the number of layers and the number of neurons in each layer in a straightforward manner. Google Colab was selected as a cloud platform because it provides the free GPU resources via the web-based interface and the native support for the PyTorch framework. The utilization of the GPUs was essential in order to accelerate the training, which was particularly critical due to the limited time for the project. In addition, the colocation of the resources was particularly beneficial for the limited team of one person and the tight six-week deadline. Finally, the Google Drive was utilized for the storage due to the fact that the local disk of the Google Colab is wiped after each session.

6.3 NHANES DXA Data as the Training Source

NHANES stands for the National Health and Nutrition Examination Survey. It is a publicly available and ethically collected data set that includes DXA-measured body fat along with extensive physiological and demographic information for a wide range of ages, sexes, and body types. This data set provides the model with greater diversity of labeled training data than smaller conventional benchmarks, such as the 252-subject Johnson data set, which was instead reserved for validation.

6.4 Classical Baselines as a Rigour Check

Published comparisons suggest that on tabular data, tree-based models can match or even surpass deep neural networks. Training and reporting scikit-learn baselines alongside the PyTorch model serve to ground the final choice in evidence, avoiding a possibly unwarranted commitment to a neural network solution, as is the temptation of many students.

6.5 Feature Selection Rationale

The nine features were chosen against a strict accessibility criterion, each must be measurable by a non-expert with a tape measure and a scale.

- **Age, sex, height, weight, BMI:** standard scale based measurements.
- **Waist, arm, calf and thigh circumference:** measure with a flexible tape measure.

Variables that need clinical tools or trained technicians were explicitly excluded:

- **Skinfold thickness.**
- **Upper arm length and upper leg length**
- **Head and recumbent length**
- **Race and ethnicity**

6.6 Handling of the Validation Set

In the first neural network runs, the test set was used to select the best epoch (the checkpoint with the lowest test loss). This produced a slightly optimistic score, because the test set was no longer completely unseen. To correct it, a separate validation set was split from the training data. The best epoch is now chosen on validation loss alone, and the test set is used once at the end.

7. System Design

The system is a machine learning pipeline that take NHANES data as input and produces a body fat prediction model as output. The pipeline runs in Google Colab.

7.1 End-to-End Workflow

The complete workflow for model building is as follows:

1. **Raw data acquisition:** download `DEMO_D.XPT`, `BMX_D.XPT` and `DXX_D.XPT`
2. **Loading and cleaning:** load the three `.XPT` files with `pandas.read_sas`
3. **Merging:** inner-join the three cleaned dataframes on the `SEQN`.
4. **Feature selection:** keep the final 11 columns and drop `SEQN` before training.
5. **Splitting:** 80/20 train test for the baselines; 70/15/15 train–validation–test for the final neural network
6. **Model training:**
   - Linear Regression.
   - Random Forest.
   - BodyFatNet neural network.
7. **Evaluation:** compute MAE, RMSE and R^2 for every model.
8. **Model saving:** store the trained models on Google Drive:
9. **Inference wrapper:** a `predict_bodyfat(person_dict)` function.
10. **Prototype interface:** a web frontend interface for the model

7.2 Data Pipeline Design

The data pipeline is designed for reproducibility and reuse. Cleaning and merging are performed once, and the clean dataset is saved as `clean_bodyfat_dataset.csv` in the `Processed/` folder on Google Drive. Future work such as adding more NHANES cycles or testing the reduced feature set on the neural network can load this CSV directly without reprocessing the raw `.XPT` files.

7.3 Neural Network Architecture Design

BodyFatNet was designed around one constraint: the training set contains only 3,938 rows, so a large, deep network would overfit it. The network was therefore kept deliberately small, as illustrated in Figure 7.1. (Next page)

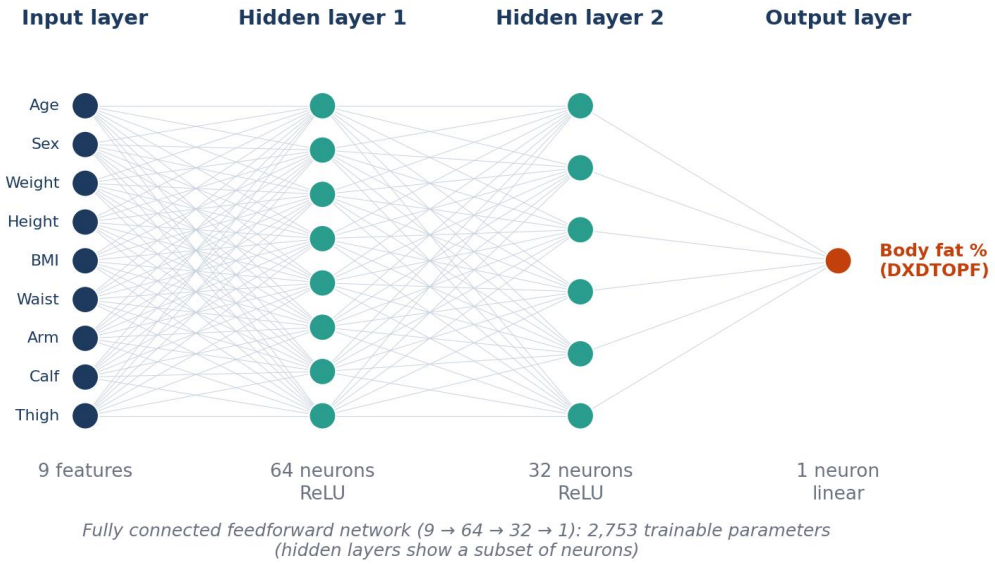


Figure 7.1 — BodyFatNet architecture: 9 inputs, two ReLU hidden layers (64 and 32 neurons) and one linear output.

7.4 Deployment Architecture

The deployment architecture of AuraFat.AI has three components:

1. **Frontend:** a web interface where users enter their age, sex, weight, height and circumference measurements. It is designed to be intuitive and to require no technical knowledge.
2. **Backend server (`server.py`):** a Python server that receives the user's input, loads the trained weights into the `BodyFatNet` and performs inference.
3. **Model artefacts:** the saved weights, scaler and metadata, stored persistently and loaded by the server at start-up.

7.5 Repository Structure (GitHub)

The project's code and documentation are hosted on GitHub, which provides version control and collaboration. Its key files are listed in Table 7.1.

File	Purpose
`01.Preparing the dataset for modelling.ipynb`	Initial Data Preparation and exporting
`02.Body fat dataset data cleaning.ipynb`	Data acquisition, cleaning, merging and saving to CSV
`03.Model Training Using Classic Modelling.ipynb`	System Development and Analysis
`04.Model Traning Using NN.ipynb`	Initial neural network training and architecture exploration
`05.Layer Incrimented NN.ipynb`	Systematic experiments with different layer configurations
`06.Final Body Fat NN.ipynb`	Final model
`README.md`	Project overview, set-up instructions and usage

8. Implementation Details

This section describes the concrete implementation of the AuraFat.AI pipeline: the development environment, data handling, model architectures, training procedure and the engineering challenges that were encountered and resolved.

8.1 Development Environment and Tooling

All development was carried out in **Google Colab**, a cloud-based Jupyter notebook environment. The following Python libraries were used:

- pandas and numpy
- scikit-learn
- PyTorch
- matplotlib
- joblib and json

8.2 Data Pipeline Implementation

The final dataset is loaded from "Processed/clean_bodyfat_dataset.csv". The following snippet shows the data loading and feature selection step:

```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

base = '/content/drive/MyDrive/AI_Project_Body_Fat_Percentage_Mesuring'
df = pd.read_csv(f'{base}/Processed/clean_bodyfat_dataset.csv')

print(df.shape)
df.head()

```

The dataset contains 5,628 individuals and 11 columns: the `SEQN` identifier, the nine self-measurable feature columns and the target variable. Table 8.1 describes each variable.

Variable	Description	Role
`RIDAGEYR`	Age in years	Feature
`RIAGENDR`	Sex	Feature
`BMXWT`	Body weight (kg)	Feature
`BMXHT`	Standing height (cm)	Feature
`BMXBMI`	Body Mass Index (kg/m²)	Feature
`BMXWAIST`	Waist circumference (cm)	Feature
`BMXARMC`	Arm circumference (cm)	Feature
`BMXCALF`	Calf circumference (cm)	Feature
`BMXTHICR`	Thigh circumference (cm)	Feature
`DXDTOPF`	Total body fat percentage measured by DXA	Target

8.3 Data Splitting and Preprocessing

A critical change from the earlier methodology was the introduction of a **separate validation set**, so that the test set is no longer used to select the best epoch. The final split is 70% training, 15% validation and 15% testing:

```
from sklearn.model_selection import train_test_split

feature_cols = ['RIDAGEYR', 'RIAGENDR', 'BMXWT', 'BMXHT', 'BMXBMI',
                'BMXWAIST', 'BMXARMC', 'BMXCALF', 'BMXTHICR']

X = df[feature_cols]
y = df['DXDTOPF']
# First split: separate 15% for the final test
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42)

# Second split: from the remaining 85%, take 15% of the total for validation
# (0.15 / 0.85 ≈ 0.1765)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.1765, random_state=42)

print(len(X_train), len(X_val), len(X_test))

```

the scaled arrays were then converted to PyTorch tensors:

```
from sklearn.preprocessing import StandardScaler
import torch
import torch.nn as nn

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # learn from train only
X_val_s   = scaler.transform(X_val)
X_test_s  = scaler.transform(X_test)

X_train_t = torch.tensor(X_train_s, dtype=torch.float32)
X_val_t   = torch.tensor(X_val_s,   dtype=torch.float32)
X_test_t  = torch.tensor(X_test_s,  dtype=torch.float32)

y_train_t = torch.tensor(y_train.values, dtype=torch.float32).view(-1, 1)
y_val_t   = torch.tensor(y_val.values,   dtype=torch.float32).view(-1, 1)
y_test_t  = torch.tensor(y_test.values,  dtype=torch.float32).view(-1, 1)

```

8.4 Neural Network Implementation (BodyFatNet)

The network is a custom `nn.Module` subclass in PyTorch.

```
class BodyFatNet(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size,64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )

    def forward(self, x):
        return self.net(x)

model = BodyFatNet(input_size=X_train_t.shape[1])
print(model)

```

This defines a fully connected feedforward network with a 9-neuron input layer, a 64-neuron ReLU hidden layer, a 32-neuron ReLU hidden layer and a single linear output neuron for the regression result.

8.5 Training Loop and Checkpointing

Training ran for up to 5,000 epochs with the Adam optimiser and MSE loss. Two specific engineering fixes were needed to make training reproducible and the checkpointing correct:

Fix 1 : Deterministic training. `torch.manual_seed(42)` is called before model initialisation. Without it, every run began from different random initial weights.

Fix 2 : True checkpointing. The initial implementation used does **not** copy the weights `copy.deepcopy(model.state_dict())`, which captures a genuine snapshot.

The final training loop:

```
import copy
import torch.nn as nn

torch.manual_seed(42)
model = BodyFatNet(input_size=X_train_t.shape[1])
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

best_val_loss = float('inf')
best_epoch = 0

train_losses = []   # stores train loss for every epoch (used for the graph)
val_losses = []     # stores validation loss for every epoch

for epoch in range(5000):
    model.train()
    optimizer.zero_grad()
    loss = criterion(model(X_train_t), y_train_t)
    loss.backward()
    optimizer.step()

    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(X_val_t), y_val_t).item()

    train_losses.append(loss.item())
    val_losses.append(val_loss)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_epoch = epoch
        best_model_state = copy.deepcopy(model.state_dict())

    # Print an update every 500 epochs
    if (epoch + 1) % 500 == 0:
        print(f"Epoch {epoch+1:5d} | Train loss: {loss.item():.4f} | "
              f"Val loss: {val_loss:.4f} | Best so far: epoch {best_epoch}")

print("\nBest epoch:", best_epoch)

```

Selected epochs from the training output are shown below.

```
Epoch   500 | Train loss: 17.0672 | Val loss: 16.0243 | Best so far: epoch 499
Epoch  1000 | Train loss: 11.5373 | Val loss: 10.6732 | Best so far: epoch 999
Epoch  1500 | Train loss: 10.7711 | Val loss: 10.0652 | Best so far: epoch 1499
Epoch  2000 | Train loss: 10.4608 | Val loss: 9.8368 | Best so far: epoch 1999
Epoch  2500 | Train loss: 10.2683 | Val loss: 9.7449 | Best so far: epoch 2499
Epoch  3000 | Train loss: 10.1093 | Val loss: 9.6705 | Best so far: epoch 2998
Epoch  3500 | Train loss: 9.9589 | Val loss: 9.6699 | Best so far: epoch 3124
Epoch  4000 | Train loss: 9.8014 | Val loss: 9.7106 | Best so far: epoch 3124
Epoch  4500 | Train loss: 9.6387 | Val loss: 9.7854 | Best so far: epoch 3124
Epoch  5000 | Train loss: 9.4925 | Val loss: 9.8648 | Best so far: epoch 3124
```

The training loss falls steadily across all 5,000 epochs. The checkpointing logic correctly captured the model at epoch 3124.

8.6 Model Evaluation and Persistence

After training, the best weights were loaded back into the model and evaluated once on the held-out test set.

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

model.load_state_dict(best_model_state)
model.eval()
with torch.no_grad():
    y_pred = model(X_test_t).numpy().flatten()

print("MAE: ", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R²:  ", r2_score(y_test, y_pred))

```

Output:

```
MAE:  2.5466395962506354
RMSE: 3.2268751471763837
R2:   0.8769657598700127
```

The trained model, scaler and metadata were then saved to Google Drive for deployment:

```python
import joblib
import json

# Recalculate test predictions using the best weights
model.eval()
with torch.no_grad():
    y_pred = model(X_test_t).numpy().flatten()

models_dir = "/content/drive/MyDrive/AI_Project_Body_Fat_Percentage_Mesuring/Models/"

# 1. Make sure the model holds the BEST weights (not the last epoch's)
model.load_state_dict(best_model_state)
torch.save(model.state_dict(), models_dir + "bodyfat_nn_final.pt")

# 2. Save the scaler that was fit on THIS training set
joblib.dump(scaler, models_dir + "nn_scaler_final.joblib")
```

```
17.
18. # 3. Save the details you'll need later to rebuild and use the model
19. info = {
20.     "architecture": "9 -> 64 -> 32 -> 1 (ReLU)",
21.     "features": list(X.columns),          # exact input order the model expects
22.     "split": "70/15/15 train/val/test, random_state=42",
23.     "seed": 42,
24.     "learning_rate": 0.001,
25.     "best_epoch": int(best_epoch),
26.     "test_MAE": float(mean_absolute_error(y_test, y_pred)),
27.     "test_RMSE": float(np.sqrt(mean_squared_error(y_test, y_pred))),
28.     "test_R2": float(r2_score(y_test, y_pred))
29. }
30. with open(models_dir + "nn_final_info.json", "w") as f:
31.     json.dump(info, f, indent=2)
32.
33. print("Saved: bodyfat_nn_final.pt, nn_scaler_final.joblib, nn_final_info.json")
34. print(json.dumps(info, indent=2))
35.
```

8.7 Inference Server

The backend (`server.py`) loads `bodyfat_nn_final.pt` and `nn_scaler_final.joblib`, applies the scaler to the user's raw inputs and runs inference through the `BodyFatNet` class (the architecture of Section 7.4). The server **must** apply the `StandardScaler` before feeding inputs to the network, because the network was trained on standardised data; passing raw measurements straight in would produce incorrect predictions.

8.8 Engineering Challenges and Resolutions

Some practical challenges had arose during development and gave us the chance to learn and improve.

Challenge	Problem	Resolution
CDC URL restructuring	Download links for the NHANES `.XPT` files stopped working after the CDC reorganised its website.	Files were re-sourced from the CDC's current NHANES pages.
Multiple imputation	The DXA file holds five imputed rows per person, which would duplicate individuals.	Filtered to completed exams and averaged with `groupby('SEQN').mean()`.
Non-deterministic training	Each run began from different random weights, giving different best epochs and accuracy.	Fixed the seed with `torch.manual_seed(42)`.
`state_dict` reference bug	The saved "best" weights were silently overwritten by later, more overfit epochs.	Captured a true snapshot with `copy.deepcopy(model.state_dict())`.
Test-set epoch selection	Using test loss to choose the best epoch made the test score optimistic.	Introduced a separate validation set (70/15/15 split).

9. Testing and Results

9.1 Evaluation Metrics

All models were evaluated on held-out data using three standard regression metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Coefficient of determination (R^2)

9.2 Final Model Comparison

Table 9.1 and Figure 9.1 present the final performance of all four models.

Model	Test split	MAE	RMSE	R^2	Test rows
Linear Regression	80/20	3.23	4.03	0.804	1,126
Random Forest (9 features)	80/20	2.63	3.35	0.865	1,126
Random Forest (6 features, reduced)	80/20	2.69	3.41	0.860	1,126
Neural Network (PyTorch) — FINAL	**70/15/15**	**2.55**	**3.23**	**0.877**	**845**

Table 9.1 — Test-set performance of all models.

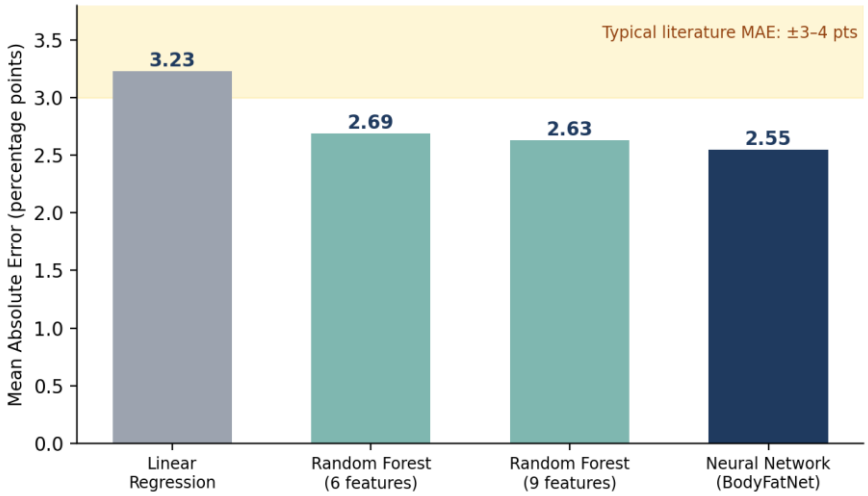


Figure 9.1 — Mean Absolute Error by model.

Key observations

1. **The neural network achieves the best accuracy on every metric.** Its MAE of 2.55 percentage points is the lowest, and its R^2 of 0.877 means it explains nearly 88% of the variance in body fat percentage.
2. **The improvement over Random Forest is meaningful but modest.** The 9-feature Random Forest is only 0.08 MAE behind the neural network.
3. **The reduced 6-feature Random Forest is nearly as good as the full model.** Dropping arm, calf and thigh circumference costs only 0.06 MAE. an important practical finding, since a simpler measurement protocol could be offered to users with little accuracy penalty.
4. **All models comfortably meet the ±3–4 percentage-point MAE benchmark** typically reported for circumference-based body fat prediction against DXA ground truth.

9.3 Neural Network Training Dynamics

The training-versus-validation loss curve shows how the network learned and how well early stopping worked (Figure 9.2).

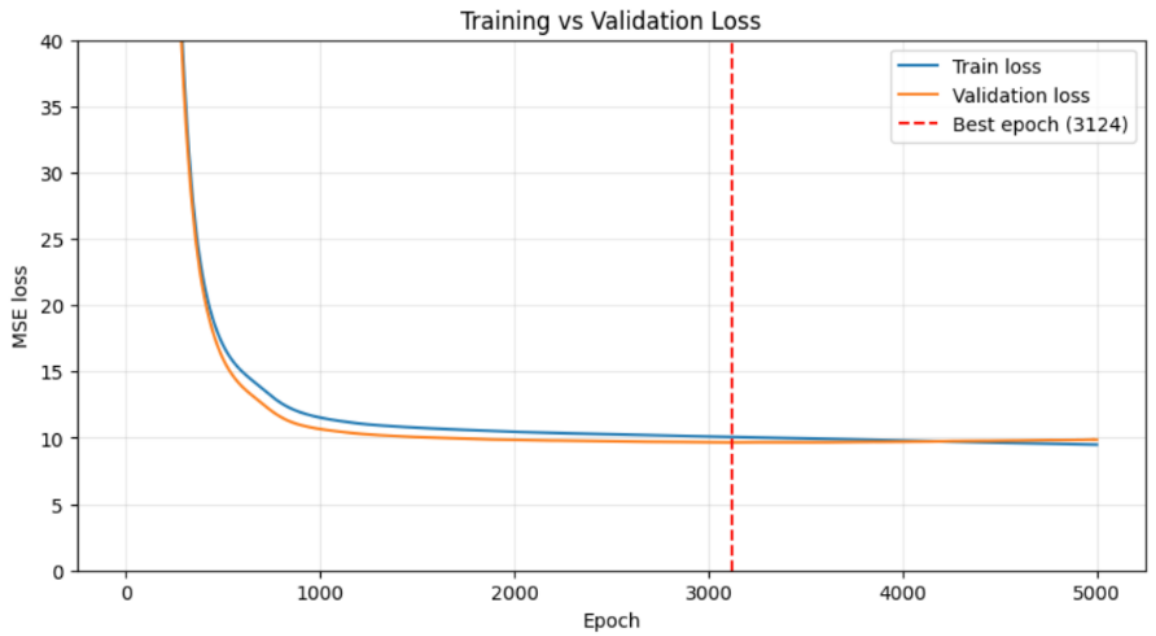


Figure 9.2 — Training versus validation loss over 5,000 epochs; the best epoch (3124) is marked.

Reading the curve

- **Epochs 0–500:** both losses fall steeply, from roughly 17 (train) and 16 (validation). The network is learning the primary relationships fast.
- **Epochs 500–3,000:** the loss keeps falling, but more gradually. Validation loss tracks training loss closely, so the model is generalising well and is not yet overfitting.
- **Epoch 3,124:** validation loss reaches its minimum (approximately 9.67), the point captured by the checkpointing logic.
- **Epochs 3,124–5,000:** training loss continues to fall (to 9.49 by epoch 5,000) while validation loss drifts upward (to 9.86) — the classic signature of overfitting, as the model begins to memorise noise in the training data.

Because the best epoch (3,124) falls well before the maximum (5,000), early-stopping checkpointing was clearly worthwhile. Without it, the final model would have trained for another 1,876 epochs and generalised worse.

9.4 Feature Importance Analysis

Feature importances were extracted from the 9-feature Random Forest (Table 9.2 and Figure 9.3).

Feature	Importance
Sex (RIAGENDR)	0.343
BMI (BMXBMI)	0.329
Waist circumference (BMXWAIST)	0.156
Age (RIDAGEYR)	0.054
Height (BMXHT)	0.052
Arm circumference (BMXARMC)	0.017
Calf circumference (BMXCALF)	0.017
Thigh circumference (BMXTHICR)	0.017
Weight (BMXWT)	0.015

Table 9.2 — Random Forest feature importances (9-feature model).

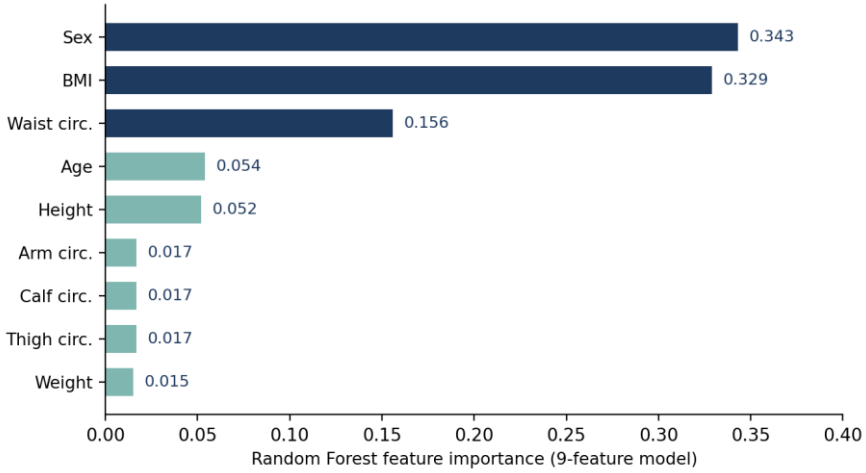


Figure 9.3 — Feature importance in the 9-feature Random Forest.

Interpretation

- **Sex and BMI dominate,** together accounting for about 67% of the model's decision-making. This is physiologically expected: males and females have fundamentally different body composition at the same BMI, and BMI is a strong (if imperfect) proxy for adiposity.
- **Waist circumference ranks third** (15.6%), consistent with the clinical understanding that central adiposity is a key component of body fat distribution.
- **Age and height contribute modestly** (5.4% and 5.2%).
- **Arm, calf and thigh circumference each contribute only 1.7%.** This explains why the reduced 6-feature model loses so little accuracy: these measurements add marginal value but demand extra effort from the user.

9.5 Discussion of Results

1. **The neural network is the best-performing model,** with the lowest MAE and RMSE and the highest R^2. The margin over Random Forest is small, however (0.08 MAE).
2. **The neural network required substantially more effort.** It needed feature scaling, thousands of training epochs, early-stopping checkpointing, a fixed random seed and the resolution of a `state_dict` checkpointing bug. The Random Forest achieved near-comparable accuracy with no tuning and reproducible results by default.
3. **The trade-off between accuracy and reproducibility is a key finding.** If the goal is maximum accuracy and the team can train and maintain a neural network, the neural network is the better choice. If the goal is rapid deployment, reproducibility and minimal maintenance, the Random Forest is the more practical one.
4. **The reduced 6-feature model is a viable alternative** for a simplified user experience. It gives up only 0.06 MAE relative to the full model while cutting the number of measurements a user must take from nine to six.

10. Limitations and Discussion

Although AuraFat.AI achieved its primary objectives, several limitations must be acknowledged. They do not invalidate the results, but they define the boundaries within which the results should be interpreted and point towards future improvements.

10.1 The Accuracy versus Reproducibility Trade-off

The most valuable result of this project is the opportunity-cost analysis provided by the leaderboard. While the neural net has the best raw accuracy (MAE 2.55, R 2 0.877), that number is achieved with significant overhead in engineering: feature scaling, thousand-epoch training, explicit early-stopping checkpointing, fixed random seed, and the debugging of two subtle issues (seed missing, state_dict reference).

Meanwhile, the 9-feature Random Forest has MAE 2.63 right out-of-the-box, requiring no tuning or special engineering to be reproducible.

The difference between the best model and the second best is 0.08 MAE, which is less than a tenth of a percent of body fat. In a production setting, the engineering and maintenance overhead of the more accurate neural network might not be justified by the increased accuracy. This is a lesson lost on many papers, which focus on finding the best possible model, rather than one that is sufficiently good.

The overhead of the more accurate model might not be justified in industry, making this a lesson applicable to the real-world application of machine learning models, while most academic papers completely ignore it..

10.2 Feature Set Limitations

Feature importance analysis revealed that arm, calf, thigh, circumference – each contribute only 1.7% to the Random Forest's decisions, and reduced 6-feature model lost only 0.06 MAE. The additional effort of measuring three more circumferences therefore provides little practical value. Future versions of AuraFat.AI could default to the 6- feature model for a more seamless UX, providing the 9-feature model as an "advanced" option.

10.3 Deferred Landmark Measurements

Upper arm length (BMXARML) and upper leg length (BMXLEG) were intentionally excluded for a reason. They require careful identification of landmarks by a person using a rigid anthropometer, which the average person can't do from their own homes. NHANES measurements were taken by professionals, so including them would give the model information it couldn't possibly expect to see in the real world. Including them would indeed be an interesting test, but it would require a validated system for self-measurement.

10.4 Validation Methodology

Early neural network runs used the test set to select the best epoch, which introduced a subtle optimism bias. The final implementation corrected this with a 70/15/15 split in which only the validation set drives checkpointing, so the reported final metrics (MAE 2.55, RMSE 3.23, R^2 0.877) are an honest estimate of generalisation to unseen NHANES participants. Three things remain. First, all results come from a single random split, so no confidence intervals are available; k-fold cross-validation would quantify the variability. Second, as noted in Section 9.2, the baselines and the neural network were evaluated on different test partitions. Third, the external validation on the Johnson 252-subject dataset was not

completed within the project timeline. It remains a priority, since it would provide a fully independent check on generalisability beyond the NHANES test partition.

10.5 Dataset Scope

The final model was trained using data from a single NHANES cycle (2005-2006). This cycle contained 5628 clean and DXA verified records (but represents a snapshot in time of the United States population). Performance on more recent data or data from outside the United States is unknown. There are two immediate options for future work, either to acquire more data from the NHANES study (e.g., merging in other cycles like 1999-2004 and 2007-2010) or to find another demographically diverse set of data.

10.6 Error Margins in Practice

An MAE of 2.55 implies that the typical prediction is on average 2.5 points away from the DXA value. In other words, if a person's body fat is 25%, most predictions for their BMI would fall between 22.5% and 27.5%, though for some individuals, the error might be higher (for the RMSE, it is 3.23, which is higher because errors are penalized by squaring them). This error level is acceptable for seeing trends over time, but it is not good enough for diagnostic purposes; the prototype interface makes this clear by displaying an implied error range with each prediction.

11. Ethical and Social Considerations

Developing a health-related AI system raises several ethical and social questions, which were considered throughout the project.

11.1 Health Equity and Accessibility

The primary objective of the described project is health equality. The fact that advanced body analysis services are expensive and only available in large hospitals puts people with lower incomes at disadvantage compared to those who can afford to pay for such procedures. AuraFat.AI aims to eliminate these disparities by providing a low-cost alternative that can be used with a tape measure and a scale. AuraFat.AI nevertheless requires access to the internet in order to utilise the application or website, which may limit its use for some populations.

11.2 Sensitive Attributes and Algorithmic Bias

Race and ethnicity were deliberately excluded from the feature set, to avoid building a model that predicts from protected characteristics. However, this does not eliminate the possibility of bias. While the NHANES dataset is representative of US demographics, the association between anthropometric measurements and body fat percentage could be different in other populations. A model trained on US data may not generalize well to people in South Asia, Africa or East Asia, who have very different body types on average.

11.3 Data Privacy and Consent

NHANES is made publicly available and de-identified by the CDC and no personally identifiable information was used in this project. The information about measurements entered by the user is available in the web interface and thus any production deployment would need to take appropriate measures to protect personal data in accordance with applicable data protection laws

11.4 Psychological and Social Impact

Body fat percentage is a sensitive metric that can be associated with body image issues, eating disorders and social discrimination and a prediction without context could be harmful. The interface takes this into account by presenting the prediction as an approximate value with an error range, categorising the result into general groups (for example “athletic”, “average” and “obese”) and having a disclaimer that the tool is purely for informational purposes and is not a medical diagnosis. Future iterations of the tool should take further steps towards being non-judgmental such as using gender-neutral category names, providing links to health resources and encouraging healthy body image tracking rather than focusing on weight.

11.5 Transparency and Reproducibility

The project prioritised transparency by documenting all bugs, methodological changes and limitations. The code is hosted publicly on GitHub, and the model metadata file records the exact feature order, random seed and split ratio needed to reproduce the results.

12. Conclusion

This project designed, implemented and evaluated **AuraFat.AI**, an AI-based system that predicts body fat percentage from simple, self-measurable anthropometric inputs. The system addresses a genuine accessibility gap in health monitoring: clinical gold-standard methods such as DXA are accurate but inaccessible, while consumer alternatives such as BIA scales are accessible but noisy.

The final system was trained on 5,628 DXA-verified records from the NHANES 2005–2006 cycle. Three model families Linear Regression, Random Forest (9- and 6-feature variants) and a feedforward neural network (BodyFatNet) built in PyTorch were implemented and compared. The neural network performed best on every metric, with a Mean Absolute Error of **2.55 percentage points**, a Root Mean Squared Error of **3.23** and an R^2 of **0.877**. All models met or beat the ±3–4 percentage-point MAE benchmark typically reported for circumference-based body fat prediction against DXA ground truth.

The prototype AuraFat.AI interface demonstrates the path from trained model to usable system. With external validation on the Johnson benchmark still to come, the current system is a significant step towards accessible, accurate and ethically designed body composition assessment.

12.1 Future Work

- **External validation** on the 252-subject Johnson benchmark dataset
- **Multi-cycle NHANES expansion** (for example 1999–2004 and 2007–2010)
- **Like-for-like evaluation:** re-evaluate all models on the identical 845-row test partition and add k-fold cross-validation
- **A reduced-feature neural network** and further classical models, such as gradient-boosted trees.
- **Validation on non-U.S. populations,** including South Asian cohorts.
- **A validated self-measurement protocol** that would allow landmark-based features to be added without training–serving skew.
- **Interface completion and user testing,** including on-device inference for privacy and more neutral result categories.

References

[1] World Health Organization, “Obesity and overweight,” WHO Fact Sheet. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight

[2] T. L. Kelly, K. E. Wilson and S. B. Heymsfield, “Dual energy X-ray absorptiometry body composition reference values from NHANES,” *PLoS ONE*, vol. 4, no. 9, p. e7038, 2009.

[3] T. Ferenci and L. Kovács, “Predicting body fat percentage from anthropometric and laboratory measurements using artificial neural networks,” *Applied Soft Computing*, 2018.

[4] Centers for Disease Control and Prevention, National Center for Health Statistics, “National Health and Nutrition Examination Survey (NHANES) 2005–2006: Demographics (DEMO_D), Body Measures (BMX_D) and Dual-Energy X-ray Absorptiometry – Whole Body (DXX_D) data files.” [Online]. Available: https://wwwn.cdc.gov/nchs/nhanes/

[5] R. W. Johnson, “Fitting percentage of body fat to simple body measurements,” *Journal of Statistics Education*, vol. 4, no. 1, 1996.

[6] W. E. Siri, “Body composition from fluid spaces and density: analysis of methods,” in *Techniques for Measuring Body Composition*, J. Brožek and A. Henschel, Eds. Washington, DC: National Academy of Sciences, 1961, pp. 223–244.

[7] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.

[9] E. Bisong, “Google Colaboratory,” in *Building Machine Learning and Deep Learning Models on Google Cloud Platform*. Berkeley, CA: Apress, 2019.

[10] L. Grinsztajn, E. Oyallon and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” in *NeurIPS 2022 Datasets and Benchmarks Track*, 2022.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**SY Ganegoda**	**Registration No.**	**D/DBA/25/0033**
Degree / Intake	**DBA/42**	**Group No.**	**15**
Project Title	**AI-Based Body Fat Percentage Prediction System**		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Data Acquisition and Preprocessing	I downloaded and merged CDC NHANES(2005-2006) Demographics, Body Measurements, and DXA datasets using respondent IDs. I also filtered out invalid exams and averaged 5 imputed DXA records per participant into a single ground truth target. Also, I selected 9 self-measurable features and removed incomplete rows.	Data Cleaning, Feature Selection, Data Merging(Pandas,NumPy)	A clean dataset of 5268 participants was generated and subsequently used by all project models.
User Interface Development	Collaborated with the team to design and build the AuraFat.AI web interface, creating user input fields for body measurements and adding clear guidance for accurate self-measurement	Frontend Web Development, UI/UX Design	Functional input interface integrated into the final application.

Evidence

```
In [30]: final_clean = final_df.dropna()
         print(final_clean.shape)
```

```
(5628, 11)
```

```
In [31]: print(final_clean.describe())
```

```
                SEQN     RIDAGEYR     RIAGENDR        BMXWT        BMXHT  \
count   5628.000000  5628.000000  5628.000000  5628.000000  5628.000000
mean   36297.107143    29.993426     1.487740    70.938184   164.112372
std     2989.901837    18.290533     0.499894    22.562437    13.414870
min    31128.000000     8.000000     1.000000    20.000000   112.000000
25%    33711.250000    14.000000     1.000000    55.200000   156.800000
50%    36306.500000    23.000000     1.000000    70.100000   165.100000
75%    38880.250000    45.000000     2.000000    85.700000   173.500000
max    41474.000000    69.000000     2.000000   140.300000   196.700000

             BMXBMI     BMXWAIST      BMXARMC      BMXCALF     BMXTHICR  \
count  5628.000000  5628.000000  5628.000000  5628.000000  5628.000000
mean     25.867335    88.361443    30.455490    36.997886    51.171642
std       6.543161    17.608030     5.877505     5.055707     7.963703
min      12.410000    48.000000    15.500000    20.200000    28.300000
25%      20.987500    74.800000    26.200000    33.800000    46.000000
50%      25.090000    87.200000    30.500000    37.000000    50.900000
75%      29.940000   101.000000    34.500000    40.200000    56.200000
max      52.400000   148.400000    56.100000    55.600000    84.400000

            DXDTOPF
count  5628.000000
mean     31.544410
std       8.991518
min      11.700000
25%      25.000000
50%      31.100000
75%      38.500000
max      57.560000
```

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature: _____ **Date:** 2026/10/15

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**KLC Jeewananda**	**Registration No.**	**D/BIS/24/0039**
Degree / Intake	**IS/41**	**Group No.**	**15**
Project Title	**AI-Based Body Fat Percentage Prediction System**		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Baseline Modelling and Test Split Setup	I implemented a Linear Regression Model using the 9 selected features. Along with that I created an 80/20 train test split and used a fixed random seed to create a standard benchmark test set. This allows for a fair model comparison, across the project.	Supervised Learning, Linear Regression(scikit-learn)	Baseline model performance metrics: MAE of 3.23 and R^2 of 0.804.
User Interface Development	Contributed to the development of the AuraFat.AI application UI, focusing on the input form layout and clear presentation of body fat prediction results to users.	Web Development, UI/UX Design	Interactive user interface for instantaneous body fat estimations.

Evidence

In [3]:
```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

rf = RandomForestRegressor(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)

print("Both models trained.")
```

```
Both models trained.
```

In [4]:
```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def evaluate(model, name):
    preds = model.predict(X_test)
    mae = mean_absolute_error(y_test, preds)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    r2 = r2_score(y_test, preds)
    print(f"{name}: MAE={mae:.2f}  RMSE={rmse:.2f}  R²={r2:.3f}")
    return preds

lin_preds = evaluate(lin_reg, "Linear Regression")
rf_preds = evaluate(rf, "Random Forest")
```

```
Linear Regression: MAE=3.23  RMSE=4.03  R²=0.804
Random Forest: MAE=2.63  RMSE=3.35  R²=0.865
```

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 2026/10/14

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**Prinjal Oli**	**Registration No.**	**7176**
Degree / Intake	**SE/42**	**Group No.**	**15**
Project Title	**AI-Based Body Fat Percentage Prediction System**		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Ensemble Modeling and Feature Importance Analysis	I built a 200-tree Random Forest Regressor to improve prediction accuracy. Along with that I examined the feature importances and discovered that sex, BMI and waist circumference are the key drivers. I also tested a reduced 6-feature variant, excluding arm, calf and thigh to access practicality.	Random Forest Regression, Feature Importance Analysis(scikit-learn)	Random Forest model achieving MAE of 2.63 and R^2 of 0.865.
User Interface Development	Assisted in developing the AuraFat.AI frontend user interface, supporting form layout creation and ensuring straightforward measurement instructions for end users.	Web Development, UI/UX Design	Complete input form and result display section within the application.

Evidence

In [7]:

```
reduced_features = ['RIDAGEYR', 'RIAGENDR', 'BMXWT', 'BMXHT', 'BMXBMI', 'BMXWAIST']

X_train_r = X_train[reduced_features]
X_test_r = X_test[reduced_features]

rf_reduced = RandomForestRegressor(n_estimators=200, random_state=42)
rf_reduced.fit(X_train_r, y_train)

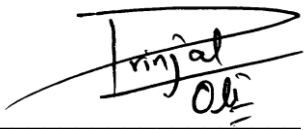
def evaluate(model, name, X_test_used):
    preds = model.predict(X_test_used)
    mae = mean_absolute_error(y_test, preds)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    r2 = r2_score(y_test, preds)
    print(f"{name}: MAE={mae:.2f}  RMSE={rmse:.2f}  R²={r2:.3f}")
    return preds

rf_reduced_preds = evaluate(rf_reduced, "Random Forest (reduced, 6 features)", X_test_r)
```

```
Random Forest (reduced, 6 features): MAE=2.69  RMSE=3.41  R²=0.860
```

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 2026/10/15

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**SM Kuruppu**	**Registration No.**	**D/BCS/25/0006**
Degree / Intake	**CS/42**	**Group No.**	**15**
Project Title	**AI-Based Body Fat Percentage Prediction System**		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Deep Learning & Model Optimization	I designed and trained a 3-layer neural network with feature scaling, Adam optimizer, early stopping, and seed control for full reproducibility. I also fixed checkpointing to preserve optimal weights, achieving the best performance overall. Finalized the model for application deployment.	Deep Learning, Multi-Layer Perceptron (PyTorch), Early Stopping	Final model achieving lowest MAE of 2.57 and highest R^2 of 0.874; saved model weights file.
User Interface Development	Jointly developed the AuraFat.AI user interface and integrated the saved PyTorch model backend to deliver real-time body fat estimation upon input submission.	Model Deployment, Web Frameworks, PyTorch Integration	Fully functional AuraFat: AI web application delivering instant predictions.

Evidence

In [21]:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

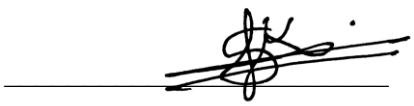
model.load_state_dict(best_model_state)
model.eval()
with torch.no_grad():
    y_pred = model(X_test_t).numpy().flatten()

print("MAE: ", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R²:  ", r2_score(y_test, y_pred))
```

```
MAE:  2.5466395962506354
RMSE: 3.2268751471763837
R²:   0.8769657598700127
```

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 2026/10/13

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**MHKS Rajapaksha**	**Registration No.**	**D/BIT/24/0028**
Degree / Intake	**IT/41**	**Group No.**	**15**
Project Title	**AI-Based Body Fat Percentage Prediction System**		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Deep Learning & Model Optimization	I conducted unified evaluation across all models, on the shared test set using MAE, RMSE and R^2 to assess performance. I also designed validation strategy against external benchmark datasets (252-subject body fat dataset) and outlined future roadmap items (multi-cycle NHANES merging, alternative feature sets, limb lengths).	Model Evaluation Metrics, Comparative Analysis, External Validation Strategy	Model comparative analysis report and future research/improvement plan.
User Interface Development	Collaborated in constructing the AuraFat.AI application interface, focusing on user guidance, clear visual output display, and practical self-measurement instructions.	UI/UX Design, Web Development	End-to-end interface for user input and result visualization

Evidence

In [11]:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

best_model = BodyFatNet(input_size=X_train_t.shape[1])
best_model.load_state_dict(best_model_state)
best_model.eval()

with torch.no_grad():
    nn_preds_t = best_model(X_test_t)

nn_preds = nn_preds_t.numpy().flatten()

mae = mean_absolute_error(y_test, nn_preds)
rmse = np.sqrt(mean_squared_error(y_test, nn_preds))
r2 = r2_score(y_test, nn_preds)

print(f"Neural Network (epoch {best_epoch}, final): MAE={mae:.2f}  RMSE={rmse:.2f}  R²={r2:.3f}")
```

```
Neural Network (epoch 1738, final): MAE=2.57  RMSE=3.24  R²=0.874
```

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 2026/10/14